

Measuring the Cost of Per-value Encryption with DBPA

[Objective](#)

[Experiment Results](#)

[String Data \(i.e. variable-sized data\) \(First and last names\)](#)

[INT32 Data \(i.e. fixed-sized data\)](#)

[Performance Breakdown](#)

[Encrypt Operation \(Variable-sized, Snappy Compression\)](#)

[Decrypt Operation \(Variable-sized, Snappy Compression\)](#)

[Key Findings](#)

[Performance of Components](#)

[Fine Tuning PME](#)

[Code-level Performance](#)

[Experimental Setup](#)

[DBPS code features to optimize performance](#)

[Clarification questions \(FAQ\)](#)

[Appendices](#)

[Appendix 1 - Raw Results](#)

[Appendix 2 - Original Implementation Measurements](#)

[Appendix 3 - Basic XOR Implementation](#)

[Appendix 4 - DBPA Local w/ no compression](#)

Objective

In this document we detail the setup and results for measuring the cost of running per-value encryption (and decryption) using the new External Encryption feature in Arrow via our new DBPA (Data Batch Protection Agent) implementation.

Experiment Results

This section shows the results of evaluating the cost of per-value encryption using DBPA (currently using XOR encryption) over AES encryption/decryption. The [experimental setup](#) section below details how the tests were run.

String Data (i.e. variable-sized data) (First and last names)

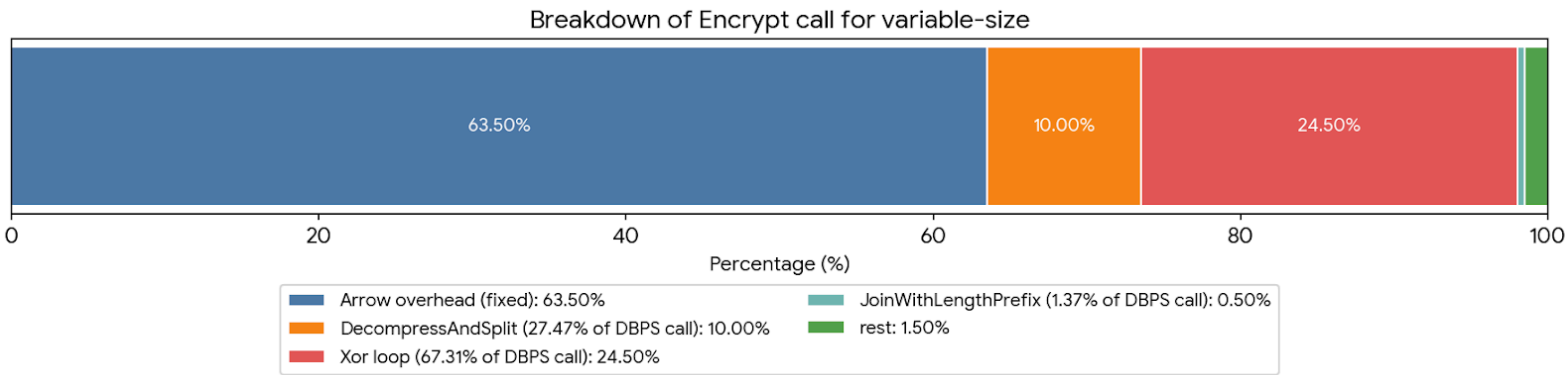
Encryption	Decryption	# of Data Pages
DBPA Δ: +1,565 ms (+76.4%) (AES: 2,049 ms) (DBPA: 3,614 ms)	DBPA Δ: +1,834 ms (+182.7%) (AES: 1,004 ms) (DBPA: 2,838 ms)	173 pages (~57.8k values/page)

INT32 Data (i.e. fixed-sized data)

Encryption	Decryption	# of Data Pages
DBPA Δ: +161 ms (+45%) (AES: 357.8 ms) (DBPA: 518.8 ms)	DBPA Δ: +222 ms (+168.2%) (AES: 131.7 ms) (DBPA: 353.7 ms)	20 pages (500k values/page)

Performance Breakdown

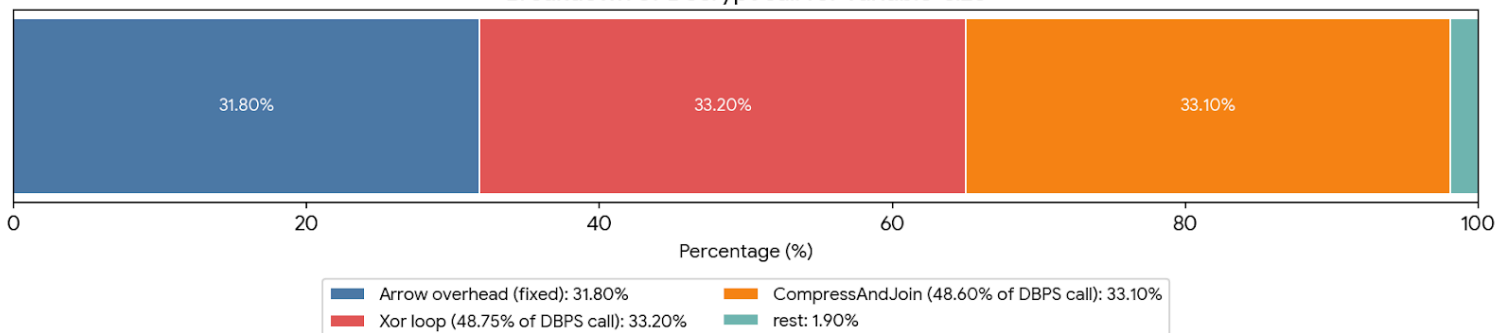
Encrypt Operation (Variable-sized, Snappy Compression)



Python::WriteEncrypt (DBPA)	3,902,791 µs	[100.0%]
└─ Parquet/Python overhead (encoding, I/O, etc.)	2,479,041 µs	[63.5%]
└─ EncryptorAdapterFactory::GetEncryptor (setup ×1)	380 µs	[0.0%]
└─ EncryptorAdapter::Make	321 µs	[0.0%]
└─ EncryptWithManagedBuffer (×162 pages, summed)	1,423,370 µs	[36.5%]
└─ InvokeExternalEncrypt	1,422,495 µs	[36.4%]
└─ Encrypt::ToPropertiesMap	965 µs	[0.0%]
└─ Encrypt::agent->Encrypt	1,401,670 µs	[35.9%]
└─ DBPS::LocalAgent::Encrypt	1,388,944 µs	[35.6%]
└─ DBPS::DecodeAndEncrypt	1,387,141 µs	[35.5%]
└─ DecompressAndSplit	388,806 µs	[10.0%]
└─ ParseTypedValues	127 µs	[0.0%]
└─ EncryptValueList	975,430 µs	[25.0%]
└─ EncryptValueList::Inner	975,172 µs	[25.0%]
└─ * XorLoop	956,960 µs	[24.5%]
└─ EncryptBlock(levels)	27 µs	[0.0%]
└─ JoinWithLengthPrefix	18,999 µs	[0.5%]
└─ Encrypt::BufferResize	1,700 µs	[0.0%]
└─ Encrypt::memcpy	3,983 µs	[0.1%]
└─ Encrypt::UpdateMetadata	307 µs	[0.0%]

Decrypt Operation (Variable-sized, Snappy Compression)

Breakdown of Decrypt call for variable-size



Python::ReadDecrypt (DBPA)	2,970,832 µs	[100.0%]
└─ Parquet/Python overhead (page assembly, I/O, etc.)	944,503 µs	[31.8%]
└─ Setup (per read_table)	4,458 µs	[0.2%]
└─ DecryptorAdapterFactory::GetDecryptor	2,401 µs	[0.1%]
└─ DecryptorAdapter::Make	2,078 µs	[0.1%]
└─ LoadAndInitializeAgent	2,057 µs	[0.1%]
└─ DecryptWithManagedBuffer (x173 pages, summed)	2,021,871 µs	[68.1%]
└─ InvokeExternalDecrypt	2,020,515 µs	[68.0%]
└─ Decrypt::agent->Decrypt	2,012,854 µs	[67.8%]
└─ DBPS::LocalAgent::Decrypt	2,000,387 µs	[67.3%]
└─ DBPS::DecryptAndEncode	1,998,795 µs	[67.3%]
└─ SplitWithLengthPrefix	~1 µs	[0.0%]
└─ * DecryptValueList	1,004,892 µs	[33.8%]
└─ DecryptValueList::Inner	1,004,665 µs	[33.8%]
└─ XorLoop	987,269 µs	[33.2%]
└─ DecryptBlock(levels)	55 µs	[0.0%]
└─ GetValueBytes	52 µs	[0.0%]
└─ * CompressAndJoin	984,090 µs	[33.1%]
└─ Decrypt::BufferResize	3,381 µs	[0.1%]
└─ Decrypt::memcpy	1,637 µs	[0.1%]

Key Findings

Performance of Components

- **The latency overhead on Arrow to interface with an external encryptor is minimal.** The performance impact of adding external encryptors on the Arrow PME module was initially unclear. In particular, Arrow's PME module preallocates memory buffers for internal encryptors, but such an optimization is impossible for external encryptors because the length of the ciphertext/plaintext cannot necessarily be estimated beforehand. This led us to add the `EncryptWithManagedBuffer` and `DecryptWithManagedBuffer` wrappers to copy results into Arrow buffers after the external encrypt/decrypt completes. Profiling shows that the overhead of these operations is only ~0.2% of the overall time involved in the encryption/decryption operations.
- **Parquet page compression significantly increases the cost of DBPA encryption/decryption.** Parquet data can be compressed as a general feature of the Arrow/Parquet write pipeline. When encrypting a compressed dataset, the DBPA per-value encryption has additional cost: because Arrow compresses data *before* handing it to an encryptor, DBPA must decompress the full page payload first, then parse and encrypt individual values, and then return the result (without recompressing, since encrypted data has low compressibility). This means that for PME use cases using DBPS per-value encryption, the chosen compression codec adds a decompress-then-discard operation overhead on the encrypt path. Profiling shows that this overhead is ~30% of the cost of DBPA's encrypt operation. Similarly, in the decrypt path, given that Arrow expects a compressed payload, DBPA must re-compress the original payload after performing the decryption of individual values. Profiling shows that this compression is around ~48% of DBPA's decrypt operation. This overhead does not exist for AES encryption, as AES does not need to interpret the payload before performing the encrypt operation.
- **The aggregated execution time of DBPA encryption and decryption operations is about the same, but the time breakdown differs significantly.** Arrow fixed overhead is significantly higher during encryption: it takes ~65% of the time of encryption, but only ~30% of time of decryption. DBPS compression is significantly higher during decryption: It takes only ~10% of the time of encryption, but notably ~33% of decryption time. This seems to have the inverse behavior or the Arrow fixed overhead. This is noted mostly as an observation, with no immediate conclusions drawn from this. XOR encryption/decryption execution times are about the same: On both encryption and decryption, the execution time of the tight for-loop for XOR encryption that iterates the elements of the input for individual processing have minor execution time differences.

Fine Tuning PME

- **Arrow configures the size of pages by payload size and not by number of elements, complicating fine-tuning of PME.** Arrow can be configured at the application level with a desired page size. This parameter is given as the size of the payload (in bytes), then internal heuristics derive the

number of elements per page. However, there is no guarantee to which degree the parameter is going to be honored, which complicates fine-tuning the number of elements per data page.

Code-level Performance

- **DBPS codepath to support variable-sized types (user strings) is more complex and less performant compared to fixed-size types.** Since the access to elements on a Parquet encoded page is done directly on the input/outputs buffers, the handling of variable-sized types is more complex. It required specialized logic to read/write elements in-place and needed the creation of indices to support random-access operations on variable-sized buffers. These resulted in more complex code and higher latency on that codepath compared to fixed-sized elements.
- **C++ vector/span allocations and creations, calls to visit methods, reading size and data are much more expensive than expected.** Since the code uses these getter methods heavily, the overall performance is impacted. Workarounds to cache values of sizes and raw pointers proved more performant. These workarounds were done in hot paths of execution to keep code readable and memory safe.

Experimental Setup

This is the setup used for gathering the performance data.

Test Dimensions

The objective of the test is to understand the cost (i.e. execution time) of running per-value encryption using DBPA, and comparing that to using AES. We measure across these dimensions:

- Data types: String (a Parquet BYTE_ARRAY), INT32.
- Operation type: encryption, decryption.

Data Utilized

- The test makes use of two types of data: Strings and INT32s
- For the test, a random sample of 10 million rows (for each data type) is used.
 - For string data, we used the “random name generator” script provided by the Protegrity team - these strings are between 10 and 20 bytes in size each and are common first name and last names.
- For each data type, the data is non-sparse (i.e. 100% of the data are non-nulls).

Execution and Measurement

- The test compares the performance of AES encryption/decryption vs that of DBPA XOR encryption/decryption.
- Tests for each data type were executed independently. One Parquet table of one column (with 10 millions rows) was created for each test.

- Tests were separated into ‘encryption’ and ‘decryption’ (i.e. each operation was measured independently).
- Parquet tables are configured to use plain encoding.
- Used the default Arrow Page Size (1MB - which only works as a hint).
- For each test:
 - The specific dataset is created in-memory. For decryption tests, the dataset is encrypted first (however this is not part of the measurement).
 - There is a ‘warmup’ phase where the operation (encrypt/decrypt) is performed 5 consecutive times.
 - Upon completing warm up, the operation (encrypt/decrypt) is repeated 10 times. The execution time of each operation (encrypt/decrypt) is recorded, then averaged out.
 - The tests are repeated for both AES and DBPA-XOR encryption algorithms.
 - Time to create `ExternalEncryptionProperties`, `ExternalDecryptionProperties`, etc is not included in the test.
- The command lines used are documented here:

 Measuring the Cost of Per-value Encryption with DBPA - RAW DATA

Hardware and Binaries Utilized

- MacBookPro, Apple M3 Pro chipset. 18GB of Memory.
- The test was run within an instance of the [Arrow Development Docker](#) container.
- The test utilized the DBPA Local Agent (`libdbpsLocalAgent.so`) for XOR, per-value encryption/decryption.
- The Arrow code, as well as the DBPA agent were compiled in their “Release” version.

DBPS code features to optimize performance

- **Zero-copy:** Introduced zero-copy read, write, and iteration paths for typed elements over contiguous byte buffers. This removed temporary vectors and reduced allocation and copies overhead during Parquet decoding and encryption/decryption.
- **Encryptor initialization:** Updated initialization to read `num_elements` directly from Parquet metadata and headers instead of inferring it from the encoded payload. This avoids an expensive setup time calculation. Also, as a nice side effect, `num_elements` becomes an invariant during `TypeBuffer` handling, so the code gets a lot simpler.
- **Encryption sequencer:** Removed compression/decompression of encrypted outputs from the sequencer pipeline. Compressing ciphertext is an anti-pattern because it may produce minimal or negative compression rates.
- **Loop-Read:** A lightweight raw pointer iterator is used to read byte spans directly from the input buffer. From the library user perspective, this makes traversal of raw byte spans very efficient, while keeping the types of the elements present.

- **Loop-Xor:** Reworked the XOR encryption path to operate directly on char* buffers, making the in-place encryption path more efficient.
- **Loop-Write:** Added GetWritableSpan support to enable direct in-place writes into the destination buffer. Also optimized the output writable buffer growth to use a single resize plus pointer-based writes, reducing reallocations and avoiding extra copies.
- **Heuristics for defragmented outputs:** When the output buffer is written in order of position, the buffer can be returned as-is as the result of the encrypt/decrypt operation, without need for defragmentation.
- **General optimizations:** Applied a set of micro-optimizations across tight loops and hot paths. Added inline method annotations, reduced additional span/vector copying at interface points. Additionally, offset the cost of calls to vector/span size() and data() by caching a lot of container sizes and pointers to data.

Clarification questions (FAQ)

Q. Does changing the Data Page size have any impact on performance?

A: For the tests described in this document, we did not make any changes to Data Page size. We used [the default](#) of 1MB. The `pagesize` parameter of [WriterProperties](#) is used as a hint for Arrow to calculate the post-compressed page size. Because this is a hint, and there are no guarantees of what the actual page size is, we decided to not include it as a dimension in the testing in order to minimize the variability among the test scenarios.

Q. How close are these tests to a real-life dataset or application?

A: The application and datasets used in these tests are synthetic (i.e. generated), and not obtained from a real-life dataset. Real life scenarios consist not only of the datasets available to consumers, but also to the consumption (i.e query) patterns and the application that generates them. Given that these may vary across a breadth of dimensions (e.g. variety of Arrow applications, specific clients, geographical locations, etc) we decided to focus on the cost of the per-value encryption/decryption per-value functionality for fully populated (i.e. non-sparse) datasets - the synthetic tests we selected should provide insight into the worst case performance scenarios.

Q. Test times are averages over 10 runs. Is that enough?

Yes. In preliminary performance tests, we encountered high variability in the execution time of the tests, in particular when the test had a low (e.g. < 50) number of runs. We managed to virtually eliminate this variability by introducing “warm up” rounds for tests. The ideal number of “warm up” rounds was 5. This number enabled us to reduce the number of actual test runs to 10, as we found minimal-to-no variability between 10 and 100 tests, for instance.

Q. Do these tests evaluate performance of the remote (i.e. network-based) external encryptor?

No. Evaluating performance for the client-server version of the external encryptor brings additional dimensions of complexity, such as the relative location of clients and the server they connect to (e.g. are they co-located in the same network? in the same country?), network connectivity type, load of the server, etc. Because of the great degree of variability in these dimensions, we felt risky to simulate some of the network conditions and decided to only produce results for the local version of DBPA.

Appendices

Appendix 1 - Raw Results

This spreadsheet contains the raw results for a breadth of scenarios which compare the AES vs DBPA encryption/decryption times.

 Measuring the Cost of Per-value Encryption with DBPA - RAW DATA

Appendix 2 - Original Implementation Measurements

This table shows the AES vs DBPA comparison in late January 2026.

AES is block-based

DBPA is value-based

Datatype	Encryption	Decryption
Variable Size (String)	DBPA: +10,000 ms (+500%) (AES: 2200 ms)	DBPA: +10,400 ms (+922%) (AES: 1,020 ms)

Appendix 3 - Basic XOR Implementation

This table shows the AES vs DBPA comparison for a simple DBPA implementation. For this, we used a simple XOR algorithm in DBPA which *always* performs block-based encryption.

AES is block-based

DBPA is ALSO block-based

Datatype	Encryption	Decryption
Variable Size (String)	DBPA: +260 ms (+12.6%) (AES: 1,100 ms)	DBPA: +410 ms (+39%) (AES: 1,050 ms)

Fixed Size (INT32)	DBPA: +160 ms (+45%) (AES: 360 ms)	DBPA: +220 ms (+155%) (AES: +370 ms)
--------------------	--	--

Appendix 4 - DBPA Local w/ no compression

These tables show the performance numbers of the fully optimized code for variable-sized and fixed-sized data. In particular, we highlight the effect that compressed data has on DBPA performance (i.e. higher cost than that of uncompressed data).

String Data (i.e. variable-sized data)

Compression Type	Encryption	Decryption
SNAPPY Compression	DBPA: +1,565 ms (+76.4%) (AES: 2,049 ms)	DBPA: +1,834 ms (+182.7%) (AES: 1,004 ms)
Uncompressed	DBPA: +988 ms (+81.57%) (AES: 1,211 ms)	DBPA: +922.8 ms (+138.6%) (AES: 665 ms)

INT32 Data (i.e. fixed-sized data)

Compression Type	Encryption	Decryption
SNAPPY Compression	DBPA: +161 ms (+45%) (AES: 357.8ms)	DBPA: +222 ms (+168.2%) (AES: 131.7 ms)
Uncompressed	DBPA: +164.7 ms (+47.2%) (AES: 348.7 ms)	DBPA: +219 ms (+158.6%) (AES: 138.3 ms)

<>